

Cloud-Native DevOps Platform

PART-3

SEETHA LAKSHMI K A P

25-06-2026

CI/CD Pipeline Implementation Using GitHub Actions and ArgoCD (GitOps)

1.Objective

The objective of this phase was to implement a complete CI/CD pipeline for the application. GitHub Actions was configured to automate Continuous Integration (CI) by building, testing, and pushing Docker images to Amazon ECR, while ArgoCD was prepared to perform Continuous Deployment (CD) using the GitOps approach.

2. Workflow Files Created

2.1) backend-ci.yml

Location

.github/workflows/backend-ci.yml

Purpose

This workflow automates the backend Continuous Integration pipeline.

Main Functions

- Trigger workflow on every push to the **main** branch.
- Checkout source code.
- Configure Python environment.
- Install required Python packages.
- Execute backend test cases.
- Configure AWS authentication using GitHub OIDC.
- Login to Amazon ECR.
- Build backend Docker image.
- Push Docker image to Amazon ECR.
- Update Kubernetes deployment manifests with the latest image tag.

Reason

Automates backend testing and Docker image creation while ensuring the latest image is available for GitOps deployment.

2.2) frontend-ci.yml

Location

.github/workflows/frontend-ci.yml

Purpose

This workflow automates the frontend Continuous Integration pipeline.

Main Functions

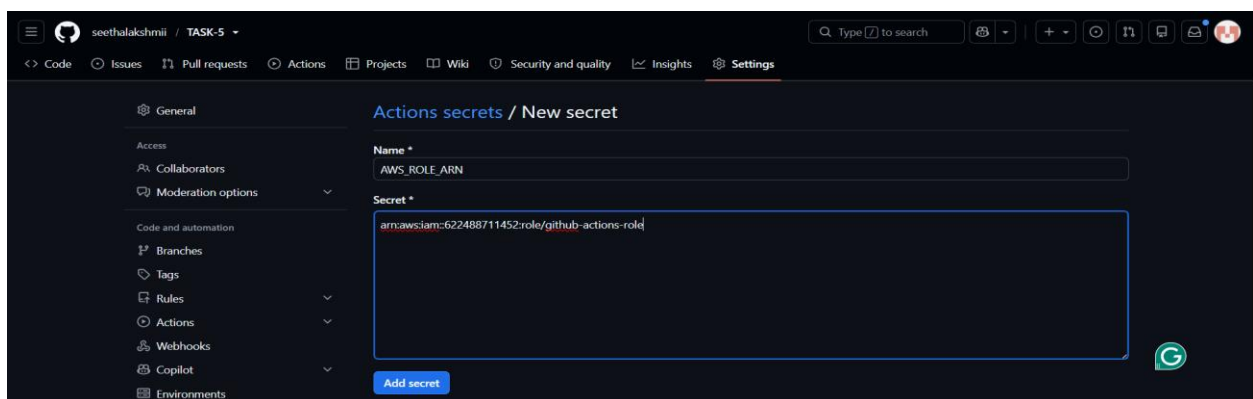
- Trigger workflow on every push to the **main** branch.
- Checkout repository.
- Configure AWS credentials.
- Login to Amazon ECR.
- Build frontend Docker image.
- Push Docker image to Amazon ECR.
- Update Kubernetes manifests with the latest image tag.

Reason

Automatically creates and publishes the latest frontend image whenever new code is pushed.

3. GitHub Repository and Secrets Configuration

- Complete project was maintained in a GitHub repository to enable CI/CD automation.
- GitHub Secrets were configured to securely store the AWS Role ARN required for GitHub Actions authentication during workflow execution.
- Instead of storing AWS Access Keys in GitHub, OpenID Connect (OIDC) authentication was configured.



4. Continuous Integration Verification

The following verification steps were performed after executing the workflows.

Check GitHub Actions workflow status

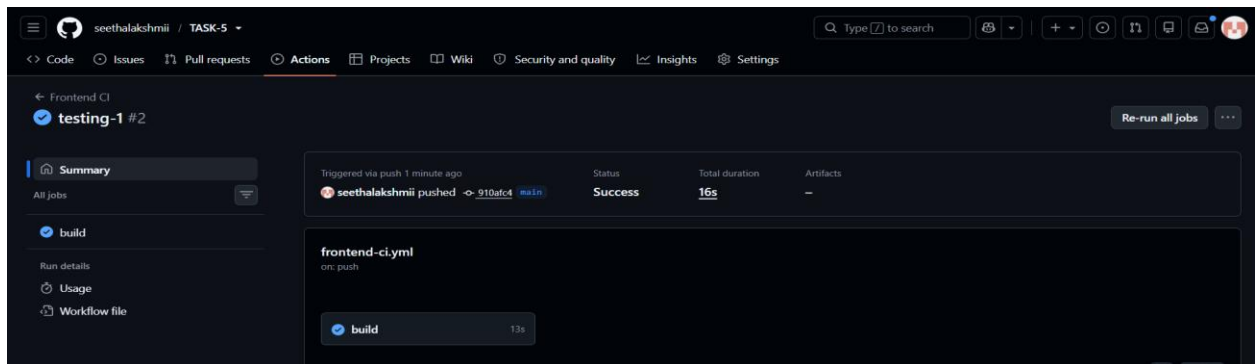
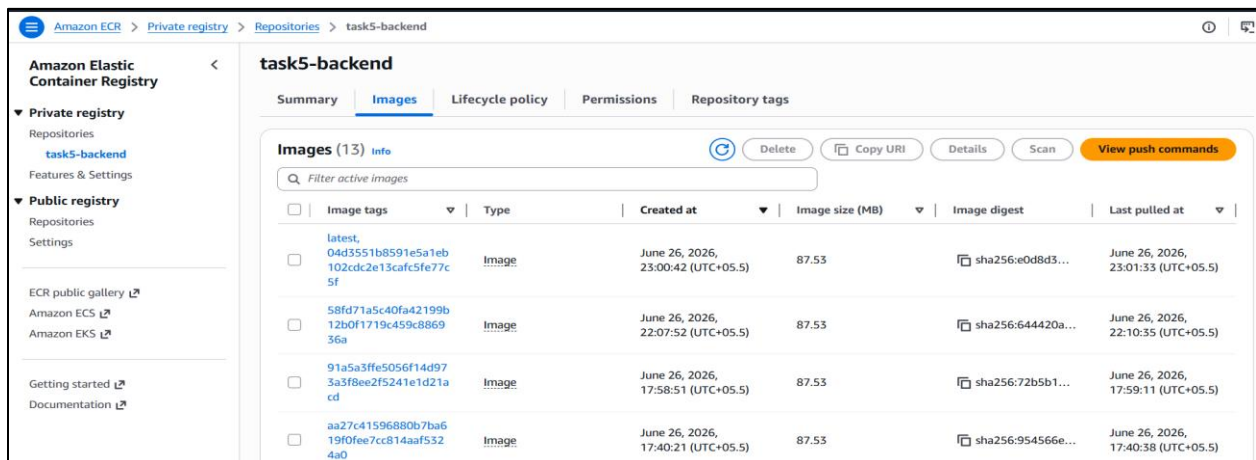
- Open GitHub Repository
- Navigate to **Actions**
- Verify successful execution of:
 - Backend CI
 - Frontend CI

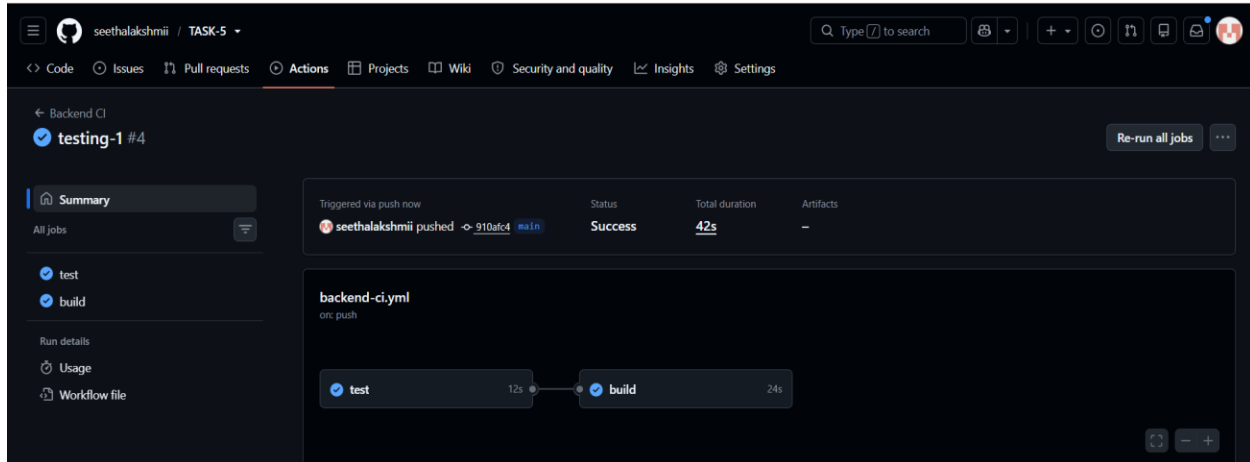
Verify Docker images in Amazon ECR:

```
aws ecr describe-images --repository-name backendaws ecr describe-images --repository-name frontend
```

List available images:

```
aws ecr list-images --repository-name backendaws ecr list-images --repository-name frontend
```





5. ArgoCD Installation

5.1) Purpose

Install ArgoCD on the Amazon EKS cluster to automate Kubernetes deployments from the GitHub repository.

Installation Commands:

Create namespace : `kubectl create namespace argocd`

Install ArgoCD:

`kubectl apply -n argocd -f https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml`

Verify all ArgoCD components: `kubectl get pods -n argocd`

```
PS C:\Users\user\Downloads\task-5> kubectl get pods -n argocd
NAME                                READY   STATUS    RESTARTS   AGE
argocd-application-controller-0     1/1     Running   0           42s
argocd-applicationset-controller-5b887868c5-9cj72  1/1     Running   0           46s
argocd-dex-server-64d89d46c8-gwtts  1/1     Running   2 (26s ago)  46s
argocd-notifications-controller-97b9678-zpn74    1/1     Running   0           45s
argocd-redis-7c89f9f856-khbcm       1/1     Running   0           44s
argocd-repo-server-7b6d58c697-scm6h  1/1     Running   0           43s
argocd-server-54569dc877-8x8tx      1/1     Running   0           43s
PS C:\Users\user\Downloads\task-5>
```

5.2) Access ArgoCD Dashboard

Purpose

Access the ArgoCD Web UI to monitor deployments and manage GitOps synchronization.

Commands

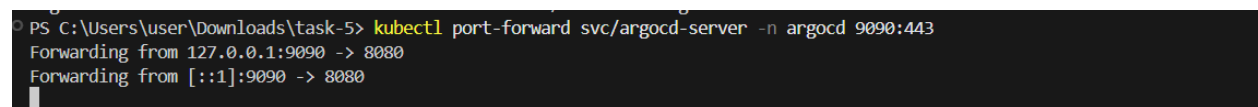
Port Forward ArgoCD Server:

```
kubectl port-forward svc/argocd-server -n argocd 9090:443
```

Retrieve Initial Password:

```
kubectl -n argocd get secret argocd-initial-admin-secret -o  
jsonpath="{.data.password}" | %  
{ [System.Text.Encoding]::UTF8.GetString([System.Convert]::FromBase64String($  
_)) }
```

Open Browser: <https://localhost:9090>



```
PS C:\Users\user\Downloads\task-5> kubectl port-forward svc/argocd-server -n argocd 9090:443  
Forwarding from 127.0.0.1:9090 -> 8080  
Forwarding from [::1]:9090 -> 8080
```

6. ArgoCD Application Files

6.1 staging-app.yaml

Location:

argocd/staging-app.yaml

Purpose

Creates the ArgoCD application for the Staging environment.

Important Configurations

- GitHub Repository
- Target Branch
- Path → k8s/staging
- Destination Cluster
- Namespace
- Auto Sync
- Self Heal
- Auto Prune

6.2 prod-app.yaml

Location:

argocd/prod-app.yaml

Purpose:

Creates the ArgoCD application for the **Production** environment.

Important Configurations

- GitHub Repository
- Target Branch
- Path → k8s/prod
- Destination Cluster
- Namespace
- Auto Sync
- Self Heal
- Auto Prune

6.3 argocd-servicemonitor.yaml

Location:

argocd/argocd-servicemonitor.yaml

Purpose:

Creates a ServiceMonitor resource for ArgoCD.

Use:

- Allows Prometheus to collect ArgoCD metrics.
- Used later for monitoring dashboards in Grafana.

7. Deploy ArgoCD Applications

Purpose

Deploy the staging and production applications to the EKS cluster.

Commands

Deploy Staging : `kubectl apply -f argocd/staging-app.yaml`

Deploy Production: `kubectl apply -f argocd/prod-app.yaml`

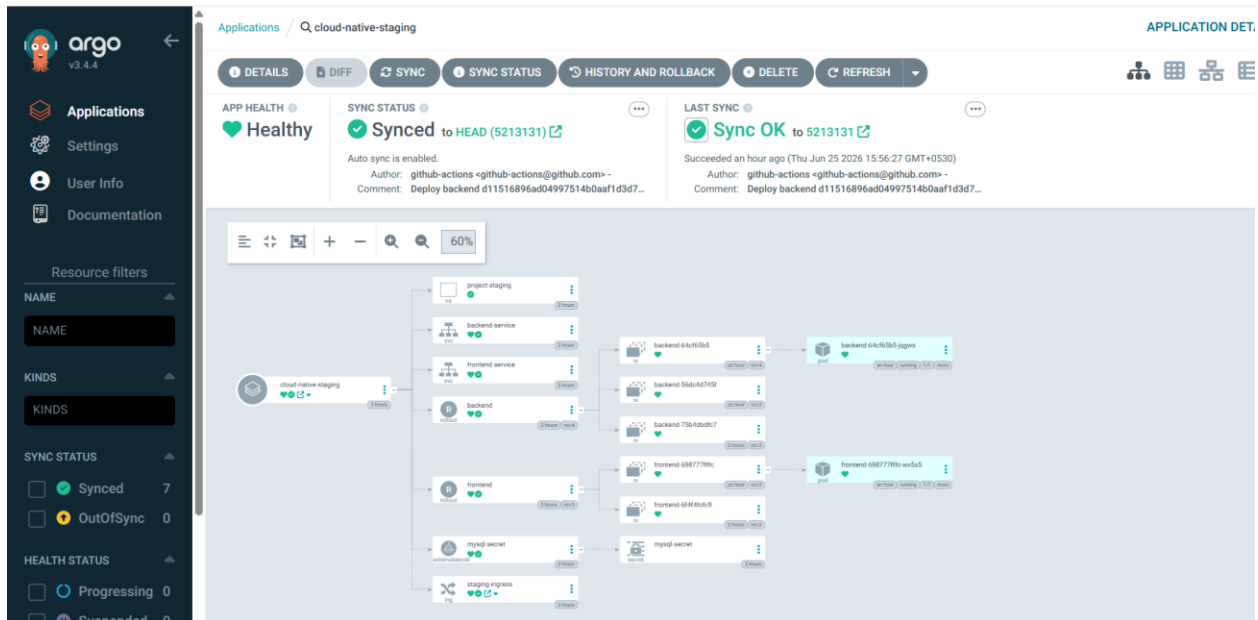
Verification : `kubectl get applications -n argocd`

```
PS C:\Users\user\Downloads\task-5> kubectl apply -f argocd/staging-app.yaml
>> kubectl apply -f argocd/prod-app.yaml
application.argoproj.io/cloud-native-staging created
application.argoproj.io/cloud-native-prod created
```

```
● PS C:\Users\user\Downloads\task-5> kubectl get application -n argocd
NAME                               SYNC STATUS   HEALTH STATUS
cloud-native-prod                  Synced        Healthy
cloud-native-staging              Synced        Healthy
○ PS C:\Users\user\Downloads\task-5> |
```

The screenshot shows the Argo CD web interface at <https://localhost:9090/applications?appNameSpace=argocd&proj=&sync=&autoSync=&health=&namespace=&cluster=&labels=&annotations=&operation=>. The interface includes a sidebar with navigation links (Applications, Settings, User Info, Documentation) and a main content area displaying a list of applications. Two applications are visible: 'cloud-native-prod' and 'cloud-native-staging'. Both applications are in a 'Healthy Synced' state, with their respective sync and health status indicators. The 'cloud-native-prod' application is associated with the 'default' project, while 'cloud-native-staging' is associated with the 'default' project. The interface also shows filters for application status (Sync Status and Health Status) and a search bar.

Application	Project	Status	Sync Status	Health Status
cloud-native-prod	default	Healthy	Synced	Healthy
cloud-native-staging	default	Healthy	Synced	Healthy



8. Accessing the Application Using Ingress

Purpose

Expose the frontend and backend applications to external users using an AWS Application Load Balancer (ALB).

Verification

Check the created ingress resources: `kubectl get ingress -A`

Retrieve the ALB DNS name

`kubectl get ingress -n staging`

`kubectl get ingress -n prod`

Access the Application

Copy the generated **ADDRESS (ALB DNS)** from the Ingress output and open it in a web browser:

`http://<ALB-DNS-Name>`

The frontend application should load successfully, and requests are routed to the backend services through the configured ingress rules.

```
PS C:\Users\user\Downloads\task-5> kubectl get ingress prod-ingress -n project-prod
NAME          CLASS  HOSTS          ADDRESS                                                                 PORTS  AGE
prod-ingress  alb    *              k8s-projectcp-prodinger-719f713b4e-1233468256.us-west-1.elb.amazonaws.com  80     161m
PS C:\Users\user\Downloads\task-5> kubectl get ingress -n project-staging
NAME          CLASS  HOSTS          ADDRESS                                                                 PORTS  AGE
staging-ingress  alb    *              k8s-projects-stagingi-dc68984b25-305943263.us-west-1.elb.amazonaws.com  80     164m
PS C:\Users\user\Downloads\task-5>
```

← → ↻ ⚠ Not secure k8s-projectp-prodingr-719f713b4e-2097579469.us-west-1.elb.amazonaws.com

Cloud Native DevOps Dashboard

Cloud Native DevOps Project..Version 2

healthy

Submit Data

Upload File to S3

No file chosen

9. Canary Deployment Using Argo Rollouts

Purpose

Deploy new application versions gradually instead of replacing all pods at once.

Features

- Progressive deployment
- Controlled traffic shifting
- Zero-downtime updates
- Safer production releases
- Automatic promotion after successful deployment

Verification

List all rollouts: `kubectl get rollouts -A`

View rollout status:

```
kubectl argo rollouts get rollout backend -n staging
```

```
kubectl argo rollouts get rollout frontend -n staging
```

Monitor rollout progress from the **ArgoCD Dashboard**.

```
PS C:\Users\user\Downloads\task-5> kubectl get rollout -n project-prod
NAME          DESIRED  CURRENT  UP-TO-DATE  AVAILABLE  AGE
backend       3        3        3           3          166m
frontend      3        3        3           3          166m
PS C:\Users\user\Downloads\task-5>
```

10. Rollback

Purpose

Restore the previous stable application version if a deployment fails or an unhealthy state is detected.

Rollback Using ArgoCD Dashboard

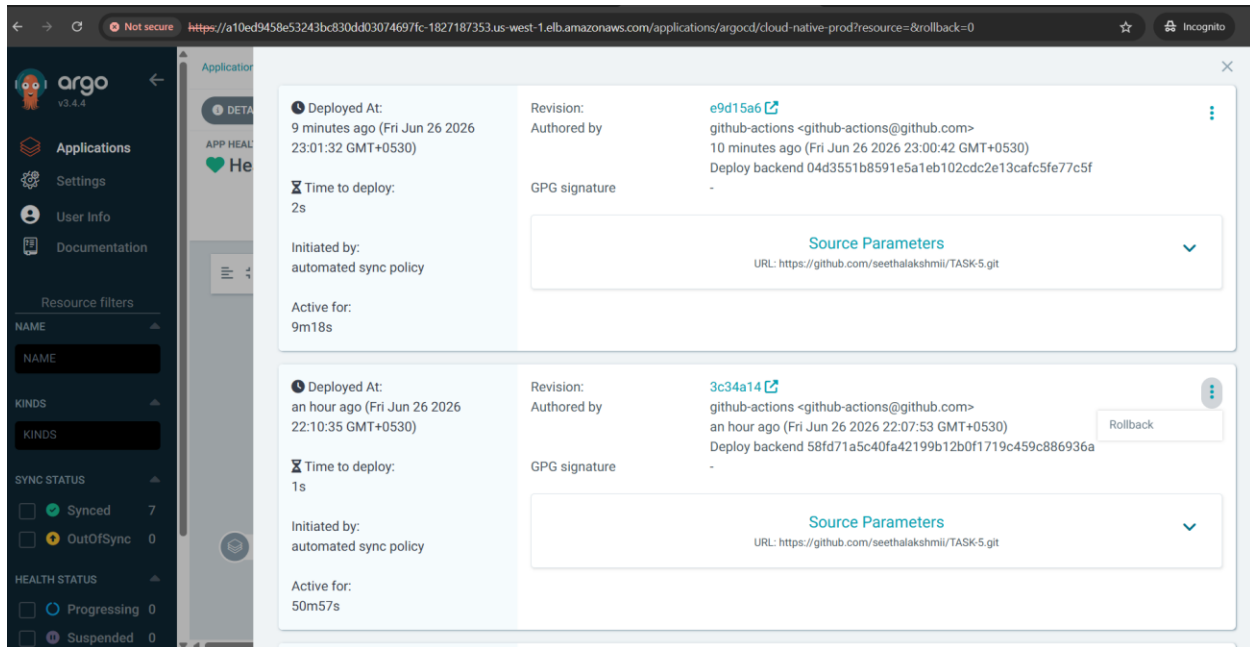
1. Open the ArgoCD Dashboard.
2. Select the required application.
3. Open the **History and Rollback** tab.
4. Select the previous successful revision.
5. Click **Rollback**.
6. Wait until the application status changes to **Healthy** and **Synced**.

Verification

Verify application status: `kubectl get applications -n argocd`

Verify rollout status: `kubectl get rollouts -A`

The previous stable revision should be restored successfully.



11. Outcome

- GitHub Actions successfully automated the CI pipeline.
- Docker images were built, tested and pushed to Amazon ECR.
- ArgoCD synchronized Kubernetes manifests automatically using the GitOps approach.
- Staging and production environments were deployed successfully on Amazon EKS.
- AWS Application Load Balancer was created through the Ingress configuration, allowing external access to the application.
- Canary deployments were successfully performed using Argo Rollouts.
- Rollback functionality was verified through the ArgoCD Dashboard.
- The complete automated CI/CD pipeline was successfully implemented and validated.